Lecture 3.2 – Choosing the Proportional Gain

Week 3: Tuning K_p for the DC Motor

Learning Objectives

By the end of this lecture, you should be able to:

- predict how changing K_p affects proportional control;
- use steady-state-error and time-constant equations to compare gains;
- recognize when a controller is too weak or too aggressive;
- explain the practical effects of PWM saturation and measurement noise;
- choose K_p using a safe experimental tuning procedure; and
- compare multiple P-controller tests using measured motor data.

1. The Tuning Problem

The proportional-control law is

$$u = K_p e.$$

The equation is simple, but we still need to choose

$$K_p.$$

If K_p is too small:

- the controller correction is weak;
- the motor may respond slowly;
- steady-state error may remain large.

If K_p is too large:

- PWM may saturate frequently;
- encoder noise may produce large control changes;
- the real system may become rough or oscillatory;
- current and mechanical stress may increase.

Controller tuning means finding a useful balance between tracking performance and practical hardware behavior.

2. Review of the Closed-Loop Equations

For the motor

$$G(s) = \frac{K}{\tau s + 1},$$

with proportional controller K_p,

$$\frac{Y(s)}{R(s)} = \frac{KK_p}{\tau s + 1 + KK_p}.$$

The ideal steady-state error for a constant reference is

$$\boxed{e_{ss} = \frac{r}{1 + KK_p}.}$$

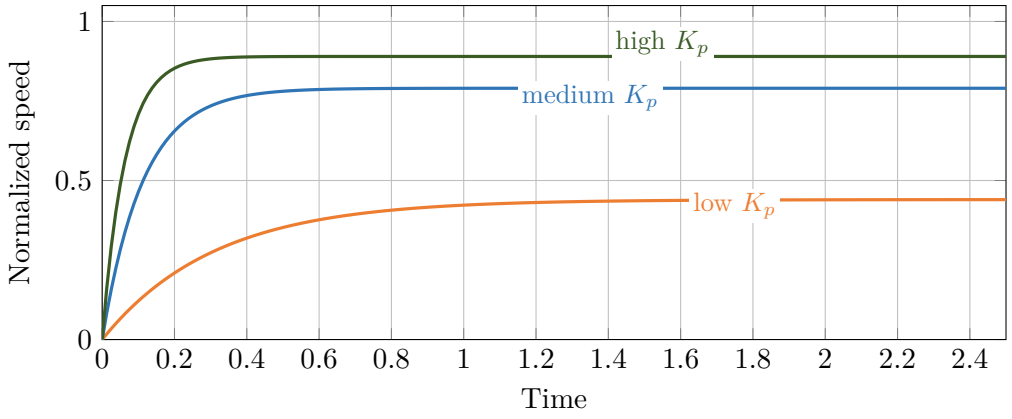
The ideal closed-loop time constant is

$$\boxed{\tau_{cl} = \frac{\tau}{1 + KK_p}.}$$

These equations help us predict what changing K_p should do.

3. Low, Medium, and High Kp

Consider the same motor with three different gains.



In the ideal first-order model, increasing K_p:

- raises the steady-state output closer to the reference;
- decreases the time constant;
- makes the response faster.

4. Numerical Comparison

Use the Week 2 example motor:

$$K = 0.775\ \frac{\text{RPM}}{\text{PWM}}, \qquad \tau = 0.55\ \text{s}.$$

K_p	KK_p	y_{ss}/r	e_{ss}/r	τ_{cl} (s)
1	0.775	0.437	0.563	0.310
2	1.550	0.608	0.392	0.216
5	3.875	0.795	0.205	0.113
10	7.750	0.886	0.114	0.063

The equations suggest that larger K_p is always better. The real motor reminds us that actuators, sensors, and sampling have limits.

5. Steady-State Error as a Percentage

The relative steady-state error is

$$\frac{e_{ss}}{r} = \frac{1}{1 + KK_p}.$$

Multiply by 100 to express it as a percentage:

$$\boxed{\%\text{ error} = \frac{100}{1 + KK_p}.}$$

For example, with

$$K = 0.775, \qquad K_p = 5,$$

$$\%\text{ error} = \frac{100}{1 + 3.875} = 20.5\%.$$

So the ideal P controller still has about 20.5% steady-state error.

6. Choosing Kp from an Error Goal

Suppose we want the ideal steady-state error to be no more than 10% of the reference.
Then

$$\frac{1}{1 + KK_p} \leq 0.10.$$

This gives

$$1 + KK_p \geq 10,$$

so

$$KK_p \geq 9.$$

Therefore,

$$\boxed{K_p \geq \frac{9}{K}.}$$

For

$$K = 0.775,$$

$$K_p \geq \frac{9}{0.775} \approx 11.6.$$

Mathematically this seems attractive.

But now consider the real PWM command.

7. The Saturation Problem

For

$$r = 150 \text{ RPM}, \qquad y(0) = 0, \qquad K_p = 11.6,$$

the initial error is

$$e(0) = 150.$$

The controller requests

$$u(0) = K_p e(0) = 11.6(150) = 1740.$$

But Arduino PWM is limited to

$$255.$$

Therefore the actual command is

$$u(0) = 255.$$

> Once the controller saturates, the simple linear closed-loop equations no longer describe the actual response exactly.

This is why theoretical gain selection must be checked on the real system.

8. What Does "Too Low" Look Like?

A K_p that is too low may produce:

- a large steady-state speed error;
- weak correction after a disturbance;
- a response that feels sluggish;
- little or no PWM saturation.

Example:
Desired speed:

$$150 \text{ RPM}$$

Measured steady speed:

$$95 \text{ RPM}.$$

Then

$$e_{ss} = 55 \text{ RPM}.$$

The controller is working, but the gain may be too small for useful tracking.

9. What Does "Too High" Look Like?

A very high K_p may cause:

- PWM to jump between large values;
- frequent saturation at 255;
- sensitivity to encoder measurement noise;
- speed fluctuations around the reference;
- rougher motor behavior.

The ideal first-order model may still predict a smooth stable response, but real hardware includes:

- sample delay;

- quantized encoder measurements;
- friction;
- motor-driver nonlinearities;
- electrical and mechanical dynamics not included in the simple model.

10. Encoder Noise and Proportional Gain

Suppose the actual speed is near the reference, but the RPM estimate changes between

$$148, \quad 153, \quad 149, \quad 152 \text{ RPM}.$$

For

$$r = 150 \text{ RPM},$$

the errors are

$$2, \quad -3, \quad 1, \quad -2 \text{ RPM}.$$

If

$$K_p = 2,$$

the controller changes by only a few PWM units.
If

$$K_p = 20,$$

the same measurement variation produces much larger control changes.

A larger proportional gain amplifies both useful error information and unwanted measurement variation.

11. Response to a Disturbance

Suppose the motor is running near its target speed.
A mechanical load is suddenly applied.

The speed decreases:

$$y \downarrow .$$

Therefore,

$$e = r - y$$

increases.
The P controller responds by increasing

$$u = K_p e.$$

The motor receives more PWM and attempts to recover.

> Feedback gives the controller a way to react to disturbances automatically.

However, with proportional control alone, the recovered speed may still remain below the reference because a nonzero error is needed to produce the extra control effort.

12. A Practical Tuning Procedure

For the Week 3 lab, use a gradual tuning method.

1. Choose a safe reference speed.
2. Start with a small K_p.
3. Run a step test.
4. Record:
   - steady-state RPM;
   - steady-state error;
   - settling behavior;
   - maximum PWM;
   - visible oscillation or roughness.
5. Increase K_p moderately.
6. Repeat the test.
7. Compare the results.

8. Stop increasing K_p if the response becomes rough, highly saturated, or unstable-looking.

> Do not search for the largest possible gain. Search for a gain that gives useful tracking while keeping the hardware behavior controlled and repeatable.

13. What Should We Measure in the Lab?

For each K_p, record:

Quantity	Why it matters
Reference RPM	desired speed
Steady-state RPM	actual final speed
Steady-state error	tracking accuracy
Approx. settling time	response speed
Maximum PWM	actuator effort/saturation
Response shape	smooth or oscillatory behavior

This lets us compare gains using data rather than only visual judgment.

14. Example Gain Comparison

Suppose $r = 150$ RPM.

K_p	Final RPM	Error	Observation
1	70	80	weak correction
3	105	45	improved tracking
6	128	22	faster, some saturation
12	138	12	aggressive, frequent saturation

These numbers are only an illustration.

The real motor data will determine the useful gain range.

15. Python Prediction

Using

$$G(s) = \frac{0.775}{0.55s + 1},$$

we can compare ideal P-control responses.

```
import numpy as np
import matplotlib.pyplot as plt

Kmotor = 0.775
tau = 0.55
R = 150

Kp_values = [1, 5, 10]
t = np.linspace(0, 2.5, 400)

for Kp in Kp_values:
    dc_gain = (Kmotor * Kp) / (1 + Kmotor * Kp)
    tau_cl = tau / (1 + Kmotor * Kp)
    y = R * dc_gain * (1 - np.exp(-t / tau_cl))
    plt.plot(t, y, linewidth=2, label=f"Kp = {Kp}")

plt.axhline(R, linestyle="--", label="Reference")
plt.grid(True)
plt.xlabel("Time (s)")
plt.ylabel("Motor Speed (RPM)")
plt.legend()
plt.show()
```

The model gives us a prediction.

The lab will show how the real system differs.

16. Practice Problem

A motor model is

$$G(s) = \frac{0.8}{0.50s + 1}.$$

The reference is

$$r = 120 \text{ RPM}.$$

Compare $K_p = 2$ and $K_p = 6$.
For each value:

1. calculate KK_p;
2. calculate e_{ss};
3. calculate y_{ss};
4. calculate τ_{cl};
5. calculate the initial ideal PWM command $u(0) = K_p r$;
6. determine whether the initial command exceeds PWM 255.

17. What P Control Can and Cannot Do

P control can:

- use feedback automatically;
- reduce speed error;
- improve response speed;
- respond to load disturbances.

P control cannot generally:

- eliminate steady-state error completely;
- avoid actuator limits;
- remove sensor noise.

The remaining steady-state error motivates the next controller:

$$\boxed{\text{PI control}}$$

which we will study in Week 4.

Connection to Lab 3

In Lab 3, you will:

1. run the motor with closed-loop proportional control;
2. choose several K_p values;
3. record reference RPM, measured RPM, error, and PWM;
4. compare steady-state error and response speed;
5. observe PWM saturation;
6. select a reasonable proportional gain for your motor.

Summary

- K_p determines how strongly the controller reacts to error.
- Larger K_p ideally reduces steady-state error.
- Larger K_p ideally decreases the closed-loop time constant.
- Real PWM is limited to 0–255.
- Large K_p can amplify encoder noise and cause rough behavior.
- Tune K_p experimentally using measured performance.
- P control generally leaves steady-state error, motivating PI control.